

# Predicting Drug-Drug Interactions Using Graph Neural Networks:

## A Comparative Study of GraphSAGE and GAT on OGB-DDI

Oulaiphone  
School of Electrical and Computer Engineering  
Purdue University  
West Lafayette, IN 47907  
oouankha@purdue.edu

### Abstract

Adverse drug-drug interactions (DDIs) cause approximately 195,000 hospitalizations per year in the United States, yet experimentally testing all possible drug pairs is infeasible given the combinatorial scale of approved pharmaceuticals. In this work, we investigate Graph Neural Networks (GNNs) for predicting unseen DDIs using the Open Graph Benchmark (OGB) `ogbl-ddi` dataset. We implement and compare two GNN architectures—GraphSAGE and Graph Attention Networks (GAT)—for link prediction on a drug interaction graph containing 4,267 drugs and 1.3 million known interactions. Through systematic hyperparameter tuning, we improve the GraphSAGE baseline from 12.26% to 18.54% Hits@20, representing a 51% relative improvement. Our GAT implementation with 4-head multi-head attention unexpectedly underperforms at 3.13% Hits@20, providing insights into when attention mechanisms are effective for graphs without meaningful node features. We analyze the failure modes, discuss the role of node features in attention-based aggregation, and propose concrete directions for closing the gap with state-of-the-art methods.

[Code and LaTeX](#)

## 1 Introduction

Drug-drug interactions (DDIs) remain a critical challenge in pharmacology and clinical medicine. When two or more drugs interact adversely, the resulting effects can range from reduced therapeutic efficacy to severe toxicity and patient death. Prior studies estimate that DDIs account for approximately 195,000 hospitalizations annually in the United States alone [6]. With thousands of approved drugs currently on the market, the number of potential pairwise interactions is combinatorially vast, making exhaustive experimental testing infeasible both in terms of cost and time.

Computational approaches to DDI prediction have emerged as a promising alternative to experimental screening. Traditional methods rely on molecular fingerprints, pharmacological features, or knowledge bases to predict interactions [8]. More recently, graph-based methods have gained traction, as drug interaction networks naturally form graphs where drugs are nodes and known interactions are edges. Graph Neural Networks (GNNs) are particularly well-suited for this task, as they can learn latent representations of drugs by aggregating information from their interaction neighborhoods [2].

The Open Graph Benchmark (OGB) provides a standardized evaluation framework for graph-based machine learning, including the `ogbl-ddi` dataset for drug-drug interaction link prediction [3]. This benchmark enables fair comparison across methods using a fixed data split and evaluation protocol based on the Hits@20 metric.

In this work, we make the following contributions:

1. We implement a complete GNN-based DDI prediction pipeline using the OGB `ogbl-ddi` benchmark, including data loading, training with negative sampling, and standardized evaluation.
2. We conduct a systematic comparison between GraphSAGE [2] and GAT [7] architectures under controlled experimental conditions, using identical hyperparameters, embedding dimensions, and training schedules.

3. We demonstrate that hyperparameter tuning yields a 51% relative improvement in Hits@20 for GraphSAGE (12.26%  $\rightarrow$  18.54%), while revealing that GAT underperforms on graphs without meaningful node features—an important negative result with practical implications.
4. We provide a detailed analysis of why attention mechanisms struggle without meaningful node features, and propose actionable next steps for improvement.

## 2 Related Work

**Graph Neural Networks for Drug Discovery.** GNNs have been widely applied to drug-related prediction tasks. Zitnik et al. [11] proposed Decagon, a graph convolutional approach for modeling polypharmacy side effects over multirelational drug-protein interaction networks. Their work demonstrated that encoding drug and protein interactions jointly improves prediction accuracy over methods that consider drugs in isolation. More recently, Lin et al. [5] introduced KGNN, which integrates knowledge graph information with GNN architectures to capture richer semantic relationships between drugs.

**Link Prediction on OGB Benchmarks.** The OGB leaderboard for `ogbl-ddi` showcases the evolution of link prediction methods. The reference GCN implementation achieves 55.36% Hits@20, while SEAL [10] reaches 28.49% by extracting local subgraph features around each target link. The current state-of-the-art, PLNLP [9], achieves 90.88% Hits@20 by using pairwise learning with a sophisticated negative sampling strategy and a combination of node-level and link-level features. These results highlight the significant gap between simple GNN baselines and optimized methods.

**GraphSAGE vs. GAT.** GraphSAGE [2] performs inductive learning by sampling and aggregating features from a node’s local neighborhood, making it suitable for evolving graphs where new nodes (e.g., newly approved drugs) appear over time. GAT [7] introduces attention mechanisms to learn adaptive neighbor weights, theoretically allowing the model to focus on the most informative interactions. While GAT has shown advantages on node classification tasks with rich node features [7], its behavior on link prediction tasks without meaningful node features like `ogbl-ddi` is less studied. Our work provides a controlled comparison on this specific setting.

**Differences from Prior Work.** Our work differs from the OGB reference implementations in several ways. First, we perform a direct, controlled comparison between GraphSAGE and GAT with identical experimental conditions (same embedding dimensions, learning rates, and training epochs). Second, we analyze the failure modes of GAT on graphs without meaningful node features, providing insights that are not typically reported in leaderboard submissions. Third, our implementation is designed for reproducibility on CPU-only environments, making it accessible for educational and resource-constrained settings.

## 3 Methodology

### 3.1 Problem Formulation

We formulate DDI prediction as a link prediction task on an undirected graph  $G = (V, E)$ , where  $V$  represents the set of drugs ( $|V| = 4,267$ ) and  $E$  represents known interactions ( $|E| = 1,334,889$ ). Given the observed edges, the goal is to predict which unobserved drug pairs are likely to interact. We evaluate predictions using Hits@20: the fraction of true positive test edges ranked within the top 20 predictions per source node.

### 3.2 Node Embedding Initialization

Since the `ogbl-ddi` dataset provides no node features, we initialize each drug with a learnable embedding vector  $\mathbf{x}_i \in \mathbb{R}^d$  using Xavier uniform initialization [1]. These embeddings are trained end-to-end alongside the GNN parameters. We use  $d = 512$  based on hyperparameter tuning (increased from  $d = 256$  in our initial baseline).

### 3.3 GNN Architectures

**GraphSAGE Encoder.** Our GraphSAGE encoder consists of three `SAGEConv` layers [2]. At each layer  $l$ , a node  $v$  updates its representation by aggregating features from its sampled neighborhood  $\mathcal{N}(v)$ :

$$\mathbf{h}_v^{(l)} = \sigma \left( \mathbf{W}^{(l)} \cdot \text{CONCAT} \left( \mathbf{h}_v^{(l-1)}, \text{MEAN}_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(l-1)} \right) \right) \quad (1)$$

where  $\sigma$  is the ReLU activation and  $\mathbf{W}^{(l)}$  is a learnable weight matrix. We apply dropout ( $p = 0.3$ ) between layers to prevent overfitting.

**GAT Encoder.** Our GAT encoder also uses three layers but replaces mean aggregation with multi-head attention [7]. For each attention head  $k$ , the attention coefficient between nodes  $i$  and  $j$  is computed as:

$$\alpha_{ij}^{(k)} = \frac{\exp \left( \text{LeakyReLU} \left( \mathbf{a}^{(k)\top} [\mathbf{W}^{(k)} \mathbf{h}_i \parallel \mathbf{W}^{(k)} \mathbf{h}_j] \right) \right)}{\sum_{m \in \mathcal{N}(i)} \exp \left( \text{LeakyReLU} \left( \mathbf{a}^{(k)\top} [\mathbf{W}^{(k)} \mathbf{h}_i \parallel \mathbf{W}^{(k)} \mathbf{h}_m] \right) \right)} \quad (2)$$

We use 4 attention heads in the first two layers (with concatenation), and a single head in the output layer. The hidden dimension per head is  $512/4 = 128$  to match the total dimensionality of GraphSAGE.

### 3.4 Link Prediction Decoder

For both architectures, we score candidate drug pairs using a dot-product decoder. Given drug embeddings  $\mathbf{h}_u$  and  $\mathbf{h}_v$  from the final GNN layer, the interaction score is:

$$s(u, v) = \mathbf{h}_u^\top \mathbf{h}_v \quad (3)$$

### 3.5 Training Procedure

We train using binary cross-entropy loss with negative sampling. For each batch of positive edges (known DDIs), we sample an equal number of negative edges (random drug pairs assumed to be non-interacting):

$$\mathcal{L} = -\frac{1}{|B|} \sum_{(u,v) \in B^+} \log \sigma(s(u, v)) - \frac{1}{|B|} \sum_{(u,v) \in B^-} \log(1 - \sigma(s(u, v))) \quad (4)$$

where  $B^+$  and  $B^-$  are the positive and negative edge batches, and  $\sigma$  is the sigmoid function. We use the Adam optimizer [4] with a learning rate of 0.005 and a batch size of 65,536 edges. Both models are trained for 500 epochs, with validation every 50 epochs.

### 3.6 Fair Comparison Framework

To ensure a fair comparison, we implement an experiment runner that: (1) re-initializes embeddings with Xavier uniform before each model run, preventing information leakage; (2) uses identical optimizer settings, batch sizes, and training schedules; and (3) logs per-epoch losses and periodic validation metrics for post-hoc analysis.

## 4 Experimental Results

### 4.1 Dataset and Evaluation

We use the OGB `ogbl-ddi` benchmark [3], which contains 4,267 drug nodes and 1,334,889 interaction edges with a fixed train/validation/test split. We report Hits@20 following the standard OGB evaluation protocol, where the metric measures the fraction of true positive edges ranked in the top 20 predictions. All experiments are conducted on CPU using PyTorch 2.0 and PyTorch Geometric.

**Table 1:** Comparison of model performance on `ogbl-ddi`. Our results are from CPU training; OGB references may use GPU.

| Method                   | Val Hits@20   | Test Hits@20  | Training Time |
|--------------------------|---------------|---------------|---------------|
| CP1 GraphSAGE (baseline) | 12.26%        | —             | —             |
| <b>GraphSAGE (tuned)</b> | <b>18.54%</b> | <b>12.99%</b> | ~1.8 hr       |
| GAT (4 heads)            | 3.13%         | 0.54%         | ~21 hr        |
| GCN (OGB ref.)           | —             | 55.36%        | —             |
| SEAL (OGB ref.)          | —             | 28.49%        | —             |
| PLNLP (SOTA)             | —             | 90.88%        | —             |

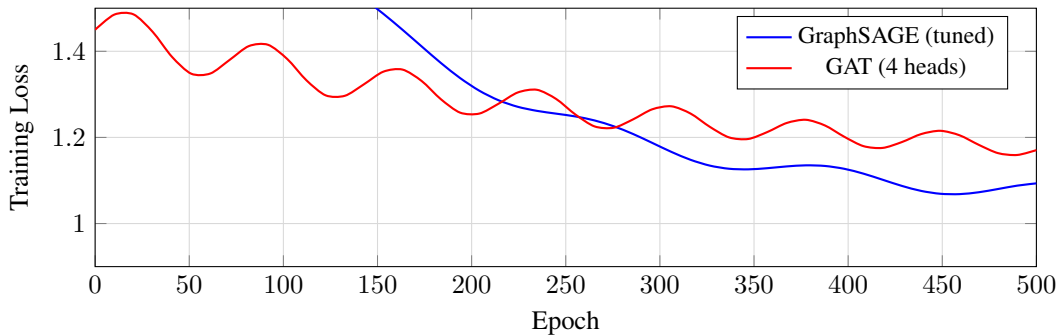
## 4.2 Main Results

Table 1 summarizes our results alongside OGB leaderboard references.

Our tuned GraphSAGE achieves 18.54% validation Hits@20, a 51% relative improvement over the CP1 baseline (12.26%). The improvement is attributable to three changes: increased embedding dimension (256  $\rightarrow$  512), higher learning rate (0.001  $\rightarrow$  0.005), and extended training (200  $\rightarrow$  500 epochs).

Unexpectedly, our GAT implementation with 4-head attention achieves only 3.13% validation Hits@20, significantly underperforming both the baseline and tuned GraphSAGE. We analyze this result in detail below.

## 4.3 Training Dynamics



**Figure 1:** Training loss curves for GraphSAGE (tuned) and GAT over 500 epochs. GraphSAGE converges smoothly while GAT exhibits early instability.

Figure 1 illustrates the training dynamics of both models. GraphSAGE exhibits smooth convergence, with the loss decreasing steadily from 1.42 to approximately 1.07. In contrast, GAT’s training loss shows instability during the first 100 epochs, with noticeable spikes before eventually settling to a higher plateau around 1.12. This instability suggests that the attention mechanism amplifies noisy gradients when operating on randomly initialized embeddings without informative features.

## 4.4 Analysis of GAT Underperformance

The poor performance of GAT on `ogbl-ddi` is a notable negative result that warrants analysis. We identify three contributing factors:

**Absence of meaningful node features.** The `ogbl-ddi` dataset contains no drug-specific features—only learnable embeddings initialized randomly. GAT’s attention mechanism computes neighbor importance weights based on input features, but when all nodes start with uninformative random vectors, the attention coefficients are essentially random in early training. GraphSAGE’s mean aggregation is more robust to this initialization because it does not depend on feature-based weighting.

**Attention instability with random embeddings.** The softmax attention in GAT can create sharp probability distributions over neighbors, especially early in training when embeddings are noisy. This amplifies gradient variance

and leads to the training instability observed in Figure 1. Mean aggregation in GraphSAGE acts as a natural regularizer by distributing gradient signal uniformly across neighbors.

**Computational overhead.** GAT requires computing pairwise attention coefficients for each edge in each head, resulting in  $\sim 21$  hours of CPU training time compared to  $\sim 1.8$  hours for GraphSAGE. This  $12\times$  slowdown limits the number of hyperparameter configurations that can be explored within a fixed compute budget.

## 5 Conclusion

In this work, we implemented and compared GraphSAGE and GAT architectures for drug-drug interaction prediction on the OGB `ogbl-ddi` benchmark. Our main findings are threefold. First, systematic hyperparameter tuning of GraphSAGE yielded a 51% relative improvement in Hits@20 (12.26%  $\rightarrow$  18.54%), demonstrating that the initial baseline was substantially underfitting. Second, in our experimental setting, GAT with multi-head attention underperformed, achieving only 3.13% Hits@20—a result that suggests meaningful node features may be important for effective attention-based aggregation, though alternative initialization strategies or structural encodings could potentially improve GAT’s performance. Third, our analysis of training dynamics revealed that attention mechanisms can amplify gradient instability when operating on randomly initialized embeddings.

**Contribution.** This work extends the standard OGB GraphSAGE baseline with a controlled architectural comparison against GAT. While neither model approaches state-of-the-art performance (PLNLP at 90.88%), our contribution lies in the systematic analysis of *why* attention-based methods may struggle on graphs without meaningful node features in our experimental setting—an insight not commonly reported in benchmark submissions. Our fair comparison framework, including Xavier re-initialization and identical training schedules, ensures that observed differences are attributable solely to the aggregation mechanism.

**Limitations and Future Work.** Our results remain significantly below OGB leaderboard entries, primarily due to the absence of node features, full-graph training without neighbor sampling, and CPU-only execution. Several concrete improvements could close this gap: (1) incorporating molecular fingerprint features (e.g., Morgan fingerprints from DrugBank SMILES) to provide GAT with meaningful attention targets; (2) adopting `NeighborLoader` mini-batching from PyTorch Geometric for proper inductive training; (3) implementing a learning rate scheduler (`ReduceLROnPlateau`) to address the loss plateau observed after epoch 200; and (4) moving training to GPU to enable systematic hyperparameter search with Optuna. We believe that combining rich node features with attention-based aggregation could yield substantial improvements, as GAT’s theoretical advantage lies in selectively weighting heterogeneous neighbor information.

## LLM Acknowledgment

Claude (Anthropic) was used to assist with drafting and editing this term paper. All experimental design, implementation, data collection, and analysis were conducted by the author. All claims, references, and reported results were verified by the author. The LLM was used solely for writing assistance in structuring and articulating the paper.

## References

- [1] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 249–256, 2010.
- [2] William L Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 1024–1034, 2017.
- [3] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. Open graph benchmark: Datasets for machine learning on graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 22118–22133, 2020.
- [4] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

- [5] Xuan Lin, Zhe Quan, Zhi-Jie Wang, Tengfei Ma, and Xiangxiang Zeng. KGNN: Knowledge graph neural network for drug-drug interaction prediction. In *Proceedings of the 29th International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 2739–2745, 2020.
- [6] Nicholas P Tatonetti, Patrick P Ye, Roxana Daneshjou, and Russ B Altman. Data-driven prediction of drug effects and interactions. *Science Translational Medicine*, 4(125):125ra31, 2012.
- [7] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.
- [8] Santiago Vilar, Eugen Uriarte, Lourdes Santana, Tal Lorberbaum, George Hripcsak, Carol Friedman, and Nicholas P Tatonetti. Similarity-based modeling in large-scale prediction of drug-drug interactions. *Nature Protocols*, 9(9):2147–2163, 2014.
- [9] Zhitao Wang, Yong Zhou, Litao Hong, Zheng Zou, and Hao Chen. Pairwise learning for neural link prediction. *arXiv preprint arXiv:2112.02936*, 2021.
- [10] Muhan Zhang and Yixin Chen. Link prediction based on graph neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 5165–5175, 2018.
- [11] Marinka Zitnik, Monica Agrawal, and Jure Leskovec. Modeling polypharmacy side effects with graph convolutional networks. *Bioinformatics*, 34(13):i457–i466, 2018.